Day 12 - Complete Lecture Notes

React Lifecycle Functions, useEffect, the key Prop, memo, useMemo and useCallback

Lecture date: 1 November | Instructor: Abhishek Purohit | Format: beginner-friendly study notes

Contents

#	Section	What you will learn
1	Session setup and recap	New demo app, the 3 class lifecycle methods already known
2	Class component: UsersList and the hidden loading bug	isLoading state, nested conditional rendering, JSX syntax errors
3	More class lifecycle methods	componentDidUpdate, componentWillUnmount, componentDidCatch
4	Lifecycle in function components	Why hooks are needed, the useEffect hook
5	useEffect as componentDidMount	Empty dependency array, fetching data on load
6	useEffect as componentDidUpdate	Dependency array with states (x, y, z)
7	Multiple useEffects and the first-render rule	Live demo with console logs and buttons
8	useEffect as componentWillUnmount	Returning a cleanup function and its rules
9	Fetching users in a function component	Why async cannot be on the effect callback
10	Rendering lists and the key prop	Virtual DOM, unique identification, performance
11	Performance optimisation and memoization	Concept with the multiplication example
12	Parent/child components and unnecessary re-rendering	Greeting and Parent demo
13	React.memo	Memoizing a whole component
14	useMemo	Memoizing the result of an expensive calculation
15	Function props, reference equality and useCallback	Counter / CounterChild demo
16	memo vs useMemo vs useCallback	Comparison and best practices
17	Doubt-clearing discussion	How to start a project, how to learn
18	Revision material	Summary, syntax, mistakes, interview Q&A, exercises, checklist
19	Completeness check	Topic-by-topic confirmation against the transcript

How to read these notes

Box colour	Meaning
Blue - FROM THE LECTURE	An important explanation or analogy given by the instructor, written in clear English.
Yellow - ADDITIONAL EXPLANATION	Extra help added by me. It is NOT something the instructor said. Read it as bonus material.
Red - COMMON MISTAKE	A frequent beginner error and how to fix it.
Green - KEY POINT	A rule or idea that is likely to be tested in MCQs or interviews.
Purple - CLARIFICATION	The transcript was auto-generated and has slips (for example, a method name said wrongly). These boxes show the correct reading.

ADDITIONAL EXPLANATION (not from the lecture)

The lecture was spoken in Hindi and English mixed. These notes are written in English and keep the instructor's order. The API URL was not spoken aloud in the transcript, so the code uses a constant named `API_URL`. For practice you can use `https://jsonplaceholder.typicode.com/users`, which returns id, name, username, email and phone fields exactly like the lecture table needs. The 3-second delay mentioned in the lecture is shown here with `setTimeout`; in the lecture it came from the instructor's own server or earlier code.

CLARIFICATION OF THE TRANSCRIPT

The lecture says `npx create-react-app`, but then runs the app on port 5173 with files `main.jsx` and `App.jsx` and the command `npm run dev`. That is the behaviour of a **Vite** React project. The commands to start the project do not change anything that is taught in this lecture, so just use whichever setup your class project already uses.

1. Session Setup and Recap

1.1 Where we are

In the last session you fetched data from an API and displayed it in a table using a **class component**. The instructor first asked whether everyone had tried that exercise at home. Today's topic is **more lifecycle functions**, followed by how to do the same job in **function components** and then several **performance optimisation** tools.

1.2 Creating the demo project

- Create a new folder for the session (the date was 1 November).
- Create a React application and name it `demo-app` (choose JavaScript when asked).
- Open the folder in VS Code, open the terminal, run `npm install` and then start the app.
- In `App.jsx` remove unused imports and `useState`, keep a very simple `App` function returning a `div`.
- In `main.jsx` remove the CSS import for now so the app stays simple.
- Create `src/UsersList.jsx`, which is the class component from the previous session.

1.3 What is a lifecycle?

A React component is born (appears on screen), lives (may update many times) and dies (is removed from screen). React lets you run your own code at each of these moments. These special moments are called **lifecycle phases**, and the methods that run at those moments are called **lifecycle methods** (or lifecycle functions).

1.4 The three class lifecycle methods already known

Method	When it runs	How many times	Use in the previous session
`constructor`	First of all, when the component is being created (mounted)	Once	Create the state that will hold the API data (`super()` must be called first)
`render`	Right after the constructor, and again after every state update	Many times	Return the user interface (UI)
`componentDidMount`	Just after the **first** render only (not after re-renders caused by state updates)	Once	Fetch data from the API when the component loads

1.5 The UsersList class component (recap)

The instructor typed the skeleton and then pasted the finished code from the previous session. The skeleton: a class named `UsersList` that `extends Component` (imported from the React package), a `constructor` that calls `super()`, and a `render` method that returns a `div` with a heading `Users List` and a `table`.

The flow of the finished component is: (1) constructor creates the state to hold the API data; (2) `render` runs and draws the table; (3) just after the first render, `componentDidMount` runs, fetches the data, and updates the state; (4) because the state changed, `render` is called again, and this time the `users` array has data, so the table shows rows.

2. Class Component UsersList and the Hidden Loading Bug

The instructor then announced: "there is one hidden bug". To see it, render `UsersList` inside `App` and run the app on port 5173. The table shows the message **Loading...** and after 3 seconds the real rows appear.

2.1 Phase 1 - The initial approach (uses `users.length` for loading)

Purpose: show a Loading message while the API call is running and show rows after the data arrives. **How the lecture's version decided this:** a conditional (ternary) check on the length of the `users` array. The assumption was: while the array is empty, the API is still being called; once data arrives, the array has rows.

Code 2.1 - UsersList, version 1 (has the hidden bug)

```
import React, { Component } from "react";

const API_URL = "https://jsonplaceholder.typicode.com/users";

class UsersList extends Component {
  constructor() {
    super();
    this.state = {
      users: [],          // starts empty
    };
  }

  componentDidMount() {
    // setTimeout only simulates the 3 second delay of a slow server
    setTimeout(() => {
      fetch(API_URL)
        .then((response) => response.json())
        .then((data) => this.setState({ users: data }))
        .catch((error) => console.log(error));
    }, 3000);
  }

  render() {
    return (
      <div>
        <h2>Users List</h2>
        <table border="1" cellPadding="6">
          <thead>
            <tr>
```

```
              <th>ID</th><th>Name</th><th>Username</th><th>Phone</th><th>Email</th>
            </tr>
          </thead>
          <tbody>
            {this.state.users.length === 0 ? (
              <tr><td colSpan="5">Loading...</td></tr>
            ) : (
              this.state.users.map((user) => (
                <tr key={user.id}>
                  <td>{user.id}</td>
                  <td>{user.name}</td>
                  <td>{user.username}</td>
                  <td>{user.phone}</td>
                  <td>{user.email}</td>
                </tr>
              ))
            )}
          </tbody>
        </table>
      </div>
    );
  }
}

export default UsersList;
```

Step-by-step explanation

- `this.state = { users: [] }` - the state that holds API data. Starting with an empty array means the table has no rows yet.
- `componentDidMount` - runs once after the first render. It calls `fetch(API_URL)`, which returns a **promise**.
- First `.then` - receives the raw `response`; `response.json()` converts the body to a JavaScript array (and itself returns another promise).
- Second `.then` - receives the real `data` array and calls `this.setState({ users: data })`. A state update makes React call `render` again.
- `.catch` - runs if the promise is rejected (for example, network failure); here the error is logged to the console.
- In `render`, the expression `condition ? A : B` is **conditional rendering**: if `users.length === 0` show the Loading row, else map each user to a table row.

Expected output: for 3 seconds the table shows `Loading...`; after that the rows with user data appear.

2.2 Phase 2 - The problem with this approach

> **FROM THE LECTURE**
> The loading decision was taken on the **length** of the users array. The assumption was that an empty array means "API is still being called". But what if the server really has no users and returns an **empty array** `[]`? The data has arrived, but the length is still 0, so the screen will show **Loading...** forever. The data will never be updated because the server has none. The user gets a **false message**. Loading should be shown only for the duration between the API call starting and the API call ending. After that, whether the data is blank or not, the UI must show the correct message (for example, "No users found").

- The length of `users` can tell us **whether we have data or not**.
- It cannot tell us **whether the API call is still running**. These are two different questions, so we need two different checks.
- The instructor noted that if we simply showed "No users found" when length is 0, then that message would also appear during the first 3 seconds, which is also wrong. So a second state is needed.

2.3 Phase 3 - Improved approach: a separate `isLoading` state

Create a new state `isLoading` that stores whether the data fetch is currently in progress. The lecture considered starting it as `false`, but chose `true`, because we fetch automatically at component load and are not waiting for a button click. When data arrives we set it to `false`. The instructor stressed using **one single `setState` call** that updates both `isLoading` and `users` together, instead of two calls.

Code 2.2 - constructor and componentDidMount with isLoading

```
constructor() {
  super();
  this.state = {
    users: [],
    isLoading: true,      // we start fetching as soon as the component mounts
  };
}

componentDidMount() {
  setTimeout(() => {
    fetch(API_URL)
      .then((response) => response.json())
      .then((data) => {
        // ONE setState call updates both values
        this.setState({ users: data, isLoading: false });
      })
      .catch((error) => console.log(error));
  }, 3000);
}
```

Now the table body needs **two nested conditions**: first check `isLoading`; only when it is `false`, check the length of `users`.

Code 2.3 - nested conditional rendering inside tbody

```
<tbody>
  {this.state.isLoading ? (
    <tr><td colSpan="5">Data is loading, please wait for a while...</td></tr>
  ) : this.state.users.length === 0 ? (
    <tr><td colSpan="5">Users not found</td></tr>
  ) : (
    this.state.users.map((user) => (
      <tr key={user.id}>
        <td>{user.id}</td>
        <td>{user.name}</td>
        <td>{user.username}</td>
        <td>{user.phone}</td>
        <td>{user.email}</td>
      </tr>
    ))
  )}
</tbody>
```

How the improved version solves the problem

Situation	isLoading	users.length	What the UI shows
API call running (first 3 seconds)	true	0	Data is loading, please wait...
API finished, server sent data	false	greater than 0	The rows
API finished, server sent `[]`	false	0	Users not found

The instructor tested both cases: with real data he saw loading for 3 seconds and then the rows; with the server returning a blank array he saw loading for 3 seconds and then **Users not found**.

2.4 The JSX syntax errors the class debugged together

While adding the conditional inside `tbody`, the editor showed an error that `tbody` has no matching closing tag. The instructor made the class find the cause. This is a very common beginner situation, so every step is noted here.

- **Why it failed:** inside JSX (the HTML-like code), anything that is JavaScript, such as a `condition ? A : B`, must be wrapped in **curly braces** `{ }`. Writing the condition directly inside `tbody` without braces is not valid.
- **Test with a smaller case:** the instructor replaced the long content with a simple `h4` message ("data arrived") and the error remained. A student (Sneha) spotted that the closing tag was written wrongly: the **forward slash must be before the tag name**, as in `</h4>`, not `<h4/>`.
- A student guessed the issue was the `return` statement. The instructor corrected this: that `return` belongs to the **map function's callback**, not to the conditional. Parentheses `( )` go with `return` only when you return multiple lines of HTML.
- Once the first condition is `isLoading`, and inside its false branch there is the second condition on `users.length`, you get **multiple conditions chained** (a nested ternary). Each branch must contain complete, correctly closed JSX.

COMMON MISTAKE

Wrong: `<h4>Data arrived<h4/>` (slash after the name). Right: `<h4>Data arrived</h4>`. Also wrong: writing `this.state.isLoading ? ... : ...` inside `<tbody>` without `{ }`. Right: `<tbody>{ this.state.isLoading ? ... : ... }</tbody>`.

KEY POINT

Conditional rendering = choosing what UI to show based on a condition, using `condition ? whenTrue : whenFalse`. Use **separate states for separate questions**: `isLoading` answers "is the request running?" and `users.length` answers "do we have data?".

3. More Class Lifecycle Methods

After `componentDidMount`, the instructor introduced three more lifecycle functions of a class component. His tip for learning them: **read the name as an English sentence** (grammar and words) and you will understand the purpose.

3.1 componentDidUpdate

Understanding the name

"Did update" means "has been updated" (past tense). So the method runs **after** the component has been updated. Students answered correctly: it means the component was re-rendered the second time (or later).

The instructor's explanation

FROM THE LECTURE

Whenever the state updates, the component **re-renders**. Just after that re-render, `componentDidUpdate` is called. So the sequence is: **state update -> render is called again -> componentDidUpdate is called.** This method runs **only on re-renders, not on the first render**. For the first render only `componentDidMount` runs. So `componentDidMount` and `componentDidUpdate` are opposites: Mount = only after the first render; Update = after every later render caused by a state update.

Where is it used? (the scoreboard example)

Imagine a scoreboard where the score can increase or decrease on the front end. Whenever the score changes on the screen, you also want to send an API call to update the score in the backend. You need to know exactly when the score is updated. Put that API call inside `componentDidUpdate`: it runs after each state update.

Code 3.1 - componentDidUpdate in a class (illustrative example of the scoreboard idea)

```
class Scoreboard extends Component {
  constructor() {
    super();
    this.state = { score: 0 };
```

```
  }

  componentDidUpdate() {
    // runs after every re-render caused by a state update, NOT after the first render
    console.log("Score changed, send API call to update backend. Score =", this.state.score);
  }

  render() {
    return (
      <div>
        <h3>Score: {this.state.score}</h3>
        <button onClick={() => this.setState({ score: this.state.score + 1 })}>+</button>
        <button onClick={() => this.setState({ score: this.state.score - 1 })}>-</button>
      </div>
    );
  }
}
```

ADDITIONAL EXPLANATION (not from the lecture)

The lecture described the scoreboard idea verbally and did not type this class. The code above is a small example added to demonstrate the same idea. Expected output: when the page loads nothing is logged by `componentDidUpdate`. Each click on + or - logs one message, because each click updates the state and causes a re-render.

3.2 componentWillUnmount

"Will unmount" means "is going to be removed". Students guessed: data leaves memory, the component exits. The instructor refined it: first a component is **mounted** (becomes visible on the screen). When the user performs some operation that takes the component **off the screen**, the component is **unmounted**. `componentWillUnmount` runs **immediately before** the component is destroyed (removed from the DOM).

FROM THE LECTURE

Java analogy: In Java you may know the `finalize` method, which runs just before an object is destroyed. The instructor joked about the last wish before the execution: the executioner asks the prisoner for a last wish. `finalize` is the last wish: tell us what you want to do before you die and we will do it. In the same way, whenever a component is about to be unmounted, any **memory cleanup or resource cleanup** work is written inside `componentWillUnmount`.

Practical example: cancelling a pending network request

The user is on the home page (home URL, home component showing). The home component made an API call to fetch users, but the call is slow (10 to 20 seconds) because of a slow network or a busy server. Meanwhile the user clicks **About Us**. The About Us screen appears and the home component is destroyed, yet the browser keeps running the API call that the home component started. That work is no longer needed because the user left the component.

- It is good practice to write the cleanup job inside `componentWillUnmount`: find the API requests that are not yet complete and cancel (close) them.
- Other cleanup examples from the lecture: closing connections and releasing resources you occupied.
- A student (Mukul) asked how this relates to the garbage collector. The instructor agreed that connections and resources you created should be cleaned by you in this method.

Code 3.2 - componentWillUnmount (illustrative)

```
class Home extends Component {
  componentDidMount() {
    console.log("Home mounted: start API call");
  }

  componentWillUnmount() {
    // called just before Home is removed from the screen
    console.log("Home will unmount: cancel pending requests, close connections");
```

```
  }

  render() {
    return <h2>Home page</h2>;
  }
}
```

> **ADDITIONAL EXPLANATION (not from the lecture)**
> In real code, cancelling a request is usually done with an `AbortController`: create it in `componentDidMount`, pass its `signal` to `fetch`, and call `controller.abort()` inside `componentWillUnmount`. The lecture explained the idea; it did not show this API.

3.3 componentDidCatch

"Catch" is a word you know from **try and catch** (exception handling). So `componentDidCatch` runs when there is an **error inside the component** (code crashes, an exception occurs). A student (Ritik) answered "exception" and the instructor confirmed. With this method you can handle the errors and exceptions occurring inside the component.

> **CLARIFICATION OF THE TRANSCRIPT**
> In the transcript the instructor sometimes says `componentDidMount` when he means `componentDidCatch` while discussing error handling (for example, "with the help of componentDidMount, you can handle the errors"). From the context it is clear he means **componentDidCatch**. Likewise, he once says `willUpdate` when he means `didUpdate`.

Real-life view: the fallback UI

You may have seen this in a mobile app or website: when something crashes you see a decent message such as "Something went wrong, please try again after some time". Some applications show a small cartoon dog; Amazon sometimes shows random animals with "Oops". That is **front-end error handling**.

> **FROM THE LECTURE**
> Without error handling, if there is an error in any one component, the **whole application suffers**: a blank screen appears and the error shows in the console. That should not happen. Imagine the page has a navbar, a counter and a footer. If the counter has an error, the navbar and footer should still render normally. In place of the counter, show a **fallback UI** such as "Something went wrong". The entire application keeps working.

It is not a straightforward feature: `componentDidCatch` works **together with the Error Boundary concept**. React provides a feature called **ErrorBoundary**. The instructor said this will be covered in a later episode. Together they let you handle errors gracefully and show a fallback UI.

3.4 Class lifecycle - complete order

Step	What happens	Lifecycle method that runs
1	Component object is created	`constructor` (first)
2	UI is drawn	`render` (just after the constructor)
3	After the first render	`componentDidMount`
4	State is updated, `render` runs again	`componentDidUpdate` (after the re-render)
5	Component is about to be destroyed (removed from the DOM)	`componentWillUnmount`
6	An error occurs during the process	`componentDidCatch`

Method	Runs on first render?	Runs on re-renders?	Typical job
`componentDidMount`	Yes (after it)	No	Fetch data when component loads

Method	Runs on first render?	Runs on re-renders?	Typical job
`componentDidUpdate`	No	Yes (after each)	Do work after a state update, e.g. sync score to backend
`componentWillUnmount`	No	No - runs just before removal	Cleanup: cancel requests, free resources
`componentDidCatch`	On error only	On error only	Handle errors, show fallback UI (with ErrorBoundary)

4. Lifecycle in Function Components

4.1 The problem

`componentDidMount`, `componentDidUpdate` and `componentWillUnmount` are **native functions of a class component**. You get them through **inheritance**: your class `extends Component`, and the `Component` class provides these methods. A function component does not inherit from anything, so if you simply write a function named `componentDidMount` inside a function component, **it will not work**.

> FROM THE LECTURE
> **Lifecycle is a native feature of a class component. It is not directly available in a function component.** To get a lifecycle in a function component you must use a **hook**. The hook is predefined in React and its name is `useEffect`.

4.2 One hook for all phases

There is no separate hook for each phase. A **single** hook, `useEffect`, can give the behaviour of `componentDidMount`, `componentDidUpdate` and `componentWillUnmount`. What changes is how you **call it**: the arguments and a small change in what the callback returns. The instructor repeated this point several times because beginners often expect three different hooks.

Class component	Function component (with the useEffect hook)
Lifecycle methods are built-in because of `extends Component`	No built-in lifecycle methods
Separate method per phase	One hook, `useEffect`, customised for each phase
Code is well organised and readable (object-oriented, clear sequence)	Slightly messier-looking syntax, but lightweight

> FROM THE LECTURE
> The instructor admitted that a class component's code is **much more readable**: you can easily recognise the sequence by reading it. Function component code looks a bit messy, but it is lightweight. "Everything has some drawback somewhere."

4.3 Syntax of useEffect

`useEffect` is imported from the React package. You always pass **two arguments**:

1. **First argument:** a **callback function** (the code to run).
2. **Second argument:** a **dependency array** (decides when the callback runs). It must always be written, even if empty.
3. Call it inside the function component, **before** the `return` statement.

Code 4.1 - the general shape of useEffect

```
import { useEffect } from "react";
```

```
useEffect(() => {
  // code to run
}, []);        // <- dependency array
```

The dependency array is explained fully in the next sections. The rule of thumb to remember first: **leave it empty for the componentDidMount behaviour**.

5. useEffect as componentDidMount

5.1 Rule

To get `componentDidMount` behaviour: call `useEffect`, pass a callback, and **leave the dependency array empty** `[]`. The callback then runs **only once, just after the first render** of the function component.

> **CLARIFICATION OF THE TRANSCRIPT**
> A function component does not have a method called `render`. The instructor therefore defines "first render" for function components as **the first time the component has returned its UI**. Just after that, the first `useEffect` runs.

5.2 Example: the UsersData function component

Purpose: fetch users data when the component loads, using a function component. **Steps in the lecture:** create `UsersData.jsx`; return a `div` with an `h2` "List of users"; import `useEffect` and `useState` from React; create a state with an empty array; call `useEffect` before `return`; do the fetch inside it.

Step A - create the state to hold the data

`useState([])` returns **two things**: the state variable and the function that updates it. The instructor named them `users` and `setUsers`. In a class you use `this.state` and `this.setState`; in a function component you use `useState`. Whichever component type you use, you create state to hold data you got from the server.

Step B - fetch with .then and .catch (first version)

`fetch` returns a promise, so you could use `async/await` or `then/catch`. For this first version the instructor used `then` and `catch`, and promised to explain the reason for avoiding `async/await` directly in `useEffect` shortly (see Section 9).

Code 5.1 - UsersData, useEffect with empty dependency array

```
import { useEffect, useState } from "react";

const API_URL = "https://jsonplaceholder.typicode.com/users";

function UsersData() {
  const [users, setUsers] = useState([]);   // users = state, setUsers = updater function

  useEffect(() => {
    fetch(API_URL)
      .then((response) => response.json())   // 1st then: promise of fetch resolved
      .then((data) => {                      // 2nd then: promise of json() resolved
        console.log(data);                   // log to prove the effect works
        setUsers(data);
      })
      .catch((error) => console.log(error)); // promise rejected
  }, []);                                    // empty array = componentDidMount behaviour

  return (
    <div>
      <h2>List of users</h2>
    </div>
  );
}

export default UsersData;
```

Line-by-line explanation

- `useState([])` - creates the state. Initial value is an empty array.
- `useEffect(callback, [])` - registers the callback. Because the array is empty, it runs once after the first render.
- `fetch(API_URL)` - sends the request and returns a promise; when resolved you get a `response` object.
- `response.json()` - reads the body as JSON. This function **itself returns a promise**, so you return it and chain another `.then`.
- Second `.then((data) => ...)` - now you have the real users array. The lecture logged it; then it would be stored with `setUsers(data)`.
- `.catch(error => ...)` - runs if any promise in the chain is rejected.

Output: open the app after replacing `UsersList` with `UsersData` inside `App.jsx`. The page shows the heading "List of users" and the browser console shows the fetched users array. That console output is the **proof** that `useEffect` with an empty array works.

KEY POINT
This is the function-component version of `componentDidMount`. Use `useEffect` **with a blank array in a function component** and `componentDidMount` **in a class component** when you want to fetch data on component load. (If you want to fetch on a button click, that is a separate thing - use a click handler.)

ADDITIONAL EXPLANATION (not from the lecture)
In development, projects created with the default React 18 template often wrap `App` in `<React.StrictMode>`. In that mode React deliberately runs mount effects **twice**, so you may see the console log appear two times. This is a development-only check and does not happen in a production build. It is not related to the lecture's behaviour.

6. useEffect as componentDidUpdate

6.1 Rule

Call `useEffect` again with a callback and a dependency array, but **this time do not leave the array empty**. Put in the array **the states whose update should trigger your code**. Whenever any of those states is updated, the callback runs. This is exactly the behaviour of `componentDidUpdate`.

6.2 The x, y, z example

Suppose the component has three states: `x`, `y` and `z` (with setter functions `setX`, `setY`, `setZ`). The initial values can be anything (number, boolean, JSON); the instructor started with 0 only as an example.

Code 6.1 - dependency array with one state

```
const [x, setX] = useState(0);
const [y, setY] = useState(0);
const [z, setZ] = useState(0);

useEffect(() => {
  // runs when x is updated
}, [x]);
```

Dependency array	The callback runs when...	It does NOT run when...
`[x]`	x is updated	y or z is updated
`[y, z]`	y or z is updated	x is updated
`[x, y, z]`	any of the three is updated	none - all are covered
`[]` (empty)	only after the first render (didMount behaviour)	any later state update

FROM THE LECTURE

That is why the second argument is called the **dependency array**: your `useEffect` is **dependent** on those state updates. Until one of those states is updated, the callback does not run. It totally depends on your requirement which state update should execute the code. You must pass the dependencies on which your logic depends.

6.3 The doubt-clearing exchange

A student (Mukul) asked for a repeat: how are mount and update implemented in `useEffect`? The summary the class arrived at:

- Dependency array **empty** -> implementation of **componentDidMount**.
- Dependency array **has states** -> implementation of **componentDidUpdate**.

7. Multiple useEffects and the First-Render Rule

The same function component can contain **multiple** `useEffect` **calls**. The instructor cleaned the previous code and created two states, `x` and `y`, then two effects:

Code 7.1 - two useEffects and two buttons

```
import { useEffect, useState } from "react";

function UsersData() {
  const [x, setX] = useState(100);
  const [y, setY] = useState(500);

  // Effect 1: empty array -> didMount behaviour
  useEffect(() => {
    console.log("DidMount called just after the first render");
  }, []);

  // Effect 2: depends on x and y -> didUpdate behaviour
  useEffect(() => {
    console.log("DidUpdate called after x or y update");
  }, [x, y]);

  const changeX = () => {
    setX(x + 10);
  };

  const changeY = () => {
    setY(y + 10);
  };

  return (
    <div>
      <h2>List of users</h2>
      <h3>x is {x} and y is {y}</h3>
      <button onClick={changeX}>Change x</button>
      <button onClick={changeY}>Change y</button>
    </div>
  );
}

export default UsersData;
```

7.1 What the class predicted, and what really happened

Before running, the instructor asked: when will the first effect run and when will the second? Correct answers: the first runs after the first render; the second runs when x or y updates; if neither updates, the second effect should not run.

Then he reloaded the page and the result **surprised the class**: both effects ran on page load, even though x and y had never been updated.

FROM THE LECTURE

The secret rule: whenever a function component renders for the **first time**, **all** the `useEffect`s are called. It does not matter whether the dependency array is empty or has states: on the first render every effect runs. You can call it an extra feature or a loophole of function components, but that is how it works.

7.2 Full behaviour table

Moment	Effect 1 (`[]`)	Effect 2 (`[x, y]`)
First render (mounting phase)	Runs	Runs
User clicks **Change x** (x updates)	Does not run	Runs
User clicks **Change y** (y updates)	Does not run	Runs
Any other re-render with x and y unchanged	Does not run	Does not run

Expected console output

```
Page loads:
  DidMount called just after the first render
  DidUpdate called after x or y update

Click "Change x" once:
  DidUpdate called after x or y update

Click "Change y" twice:
  DidUpdate called after x or y update
  DidUpdate called after x or y update
```

The instructor clicked repeatedly and showed that the count of the DidUpdate log increases each time, while the DidMount log stays at one. The screen text shows the initial values 100 and 500, and they change as the buttons are clicked.

KEY POINT

During the mounting phase (first render), both useEffects are called. Later, on a state update, only the useEffect with the matching dependencies is called. The empty-array effect never runs again.

7.3 Why have a separate empty-array effect? (student question)

A student (Abhay) asked: if a dependency-array effect also runs on the first render, why do we need the empty-array one? He suggested using a flag and an `if` condition to find out whether it is the first render. The instructor said you can definitely do that, but if a piece of logic should run **only and only on the first render and never after a state update**, then counting renders and writing `if` checks just makes the code messy. It is cleaner to keep one effect with a blank array and another effect with dependencies.

8. useEffect as componentWillUnmount

8.1 Rule

Again call `useEffect` with a callback and a dependency array. For `componentWillUnmount` the array **does not matter** (it can be empty or contain dependencies). The thing that matters is a **change inside the callback**: it must **return a function**. That returned function gives the behaviour of `componentWillUnmount` and is called **just before the component is destroyed**.

Code 8.1 - returning a cleanup function

```
useEffect(() => {
  console.log("Effect body: runs after first render (empty array)");

  // this returned function = componentWillUnmount
  return () => {
```

```
    console.log("Cleanup: runs only when the component is being destroyed");
  };
}, []);
```

8.2 Important points explained by the instructor

- **No separate useEffect is needed** for will-unmount. Add the returned function into any existing `useEffect`. Generally it is written in the empty-dependency one because it is less complex than the others.
- **Only the returned function runs on destroy.** Do not think the whole `useEffect` runs when the component unmounts. The code above the `return` runs according to the dependency array (after first render and/or on dependency updates).
- Inside the returned function you write the **cleanup job**: the same work you would do in `componentWillUnmount` of a class.
- **Returning is mandatory.** If you do not write `return`, the code is not treated as the will-unmount function.
- The effect callback can return **only a function**. Returning anything else is not allowed.

> **FROM THE LECTURE**
> Instructor's demonstration: he wrote `return 7;` inside the effect callback and React showed an error saying that a `useEffect` must not return anything besides a function, which is used for cleanup, and that 7 was returned. "You cannot return a number, a string, a boolean or anything else. Only a function." He also wrote a comment: only this returned function will be executed on component destroy.

Code 8.2 - incorrect: returning a non-function value

```
useEffect(() => {
  // ...
  return 7;          // ERROR: useEffect must not return anything besides a function
}, []);
```

8.3 The three variations side by side

The instructor wrote a quick comparison in code form. The three ways to call `useEffect`:

Code 8.3 - summary of the three variations

```
// 1) componentDidMount  -> callback + EMPTY dependency array
useEffect(() => {
  // runs once, after the first render
}, []);

// 2) componentDidUpdate -> callback + dependency array WITH states
useEffect(() => {
  // runs on first render AND whenever x or y is updated
}, [x, y]);

// 3) componentWillUnmount -> callback that RETURNS a function
useEffect(() => {
  // ...
  return () => {
    // cleanup: runs just before the component is destroyed
  };
}, []);       // array can be empty or have dependencies
```

Lifecycle phase	Class component	Function component with useEffect
componentDidMount	Method `componentDidMount()`	`useEffect(cb, [])`
componentDidUpdate	Method `componentDidUpdate()`	`useEffect(cb, [dep1, dep2])`
componentWillUnmount	Method `componentWillUnmount()`	Function **returned** from the `useEffect` callback
componentDidCatch	Method `componentDidCatch()`	No hook shown in this lecture (Error Boundary comes later)

ADDITIONAL EXPLANATION (not from the lecture)

Honest accuracy note: the last row is not from the lecture. The lecture did not show a hooks replacement for `componentDidCatch` and said the ErrorBoundary topic will come later.

9. Fetching Users in a Function Component (async/await)

The instructor then repeated the earlier class job in a function component: **fetch users at component load and show them in a table.** This time he wanted to use `async/await`, and that led to an important rule.

9.1 Phase 1 - the incorrect approach: async on the effect callback

Inside the empty-array `useEffect`, put a `try/catch` and write `await fetch(...)`. Using `await` requires the surrounding function to be marked `async`, so the natural thing is to write `useEffect(async () => { ... }, [])`.

Code 9.1 - incorrect: async effect callback

```
useEffect(async () => {                       // WRONG
  try {
    const response = await fetch(API_URL);
    const data = await response.json();
    setUsers(data);
  } catch (error) {
    console.log(error);
  }
}, []);
```

9.2 Phase 2 - why it is wrong

FROM THE LECTURE

An `async` function **always returns a promise**. The `async` keyword on a function is a declaration: "this function returns a promise". But React's rule for `useEffect` is: **the effect callback must not return anything besides a function.** By writing `async` on the callback you declare the opposite of the rule: you are saying the effect returns a promise. That is why the editor shows a warning: *Effect callbacks are synchronous to prevent race conditions. Put the async function inside.* The language may be hard to understand now; just remember that the effect callback is allowed to return only a function.

9.3 Phase 3 - the solution: a separate async function inside the effect

Move the fetching code into **another function**, named `fetchUsersData`, mark that function `async`, and **call** it from the effect. The effect callback itself stays normal (not async) and returns nothing.

Code 9.2 - UsersData with async function and users table

```
import { useEffect, useState } from "react";

const API_URL = "https://jsonplaceholder.typicode.com/users";

function UsersData() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    async function fetchUsersData() {
      try {
        const response = await fetch(API_URL);   // promise of fetch
        const data = await response.json();      // promise of json()
        setUsers(data);                          // store users in state
      } catch (error) {
        console.log(error);
      }
    }

    fetchUsersData();    // call it, effect itself is NOT async
  }, []);                // empty array: runs after the first render
```

```
    return (
      <div>
        <h2>List of users</h2>
        <table border="1" cellPadding="6" cellSpacing="0">
          <thead>
            <tr>
              <th>ID</th>
              <th>Name</th>
              <th>Username</th>
              <th>Phone</th>
              <th>Email</th>
            </tr>
          </thead>
          <tbody>
            {users.map((user) => (
              <tr key={user.id}>
                <td>{user.id}</td>
                <td>{user.name}</td>
                <td>{user.username}</td>
                <td>{user.phone}</td>
                <td>{user.email}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    );
  }

  export default UsersData;
```

Explanation

- `fetchUsersData` is declared with `async`, so `await` can be used inside it. It holds the same logic as the class `componentDidMount`.
- `await fetch(API_URL)` waits for the promise and gives the `response`. `await response.json()` waits again and gives the users array.
- `setUsers(data)` updates the state. The component re-renders and the table body is filled.
- `users.map(...)` converts the array of users into an array of `<tr>` rows. For each user, five `<td>` cells show `id`, `name`, `username`, `phone` and `email`.
- The table header has five fixed columns: ID, Name, Username, Phone, Email.
- Border, cell padding and cell spacing can be added to the table to make it look better (the lecture mentioned them).

Expected output: the heading "List of users" and the table with all rows once the data arrives. Execution order: first render (empty table) -> effect runs -> `fetchUsersData()` called -> data arrives -> `setUsers` -> re-render with rows.

> **CLARIFICATION OF THE TRANSCRIPT**
> The transcript says "one is the tbody and another will be tbody" when creating the table. The instructor means **thead** (for the header row) and **tbody** (for the rows).

> **KEY POINT**
> **Rule:** never put `async` on the `useEffect` callback. Create an inner (or outer) async function and call it. Same job as the class component, now done in a function component. Instructor's preference: use async/await rather than long `then/catch` chains because it is easier to read and maintain.

10. Rendering Lists and the key Prop

When the table rows were rendered by `map`, the browser console showed an error/warning: **"Each child in a list should have a unique key prop."** This appears whenever you render a list, either in a table or anywhere else in the UI.

10.1 The rule

> **KEY POINT**
> **Rule in React:** every element of a list rendered by your component must have a `key` **prop**, and the value of the key must be **unique** among the elements of that list. It applies to table rows and to any other HTML elements you render in a list, in both class and function components.

Code 10.1 - adding key to list items

```
{users.map((user) => (
  <tr key={user.id}>          {/* key given a unique value: user.id */}
    <td>{user.id}</td>
    <td>{user.name}</td>
  </tr>
))}
```

After adding `key={user.id}` and reloading, the console error disappears.

10.2 Why does React need a key?

React uses the key to uniquely identify each element inside a list. To see why this matters, recall the **virtual DOM**.

Background: virtual DOM (explained in the previous session)

React creates its own virtual DOM. When a component is about to render, React **compares the previous DOM with the latest DOM** and finds the differences. Only the sections that changed are re-rendered / updated on the screen.

Without a key

> **FROM THE LECTURE**
> Even though the virtual DOM works normally, **for a list without keys**, if React finds any difference, instead of updating only the changed element it **recreates the entire list**. React does not know which element had which unique identity, so it cannot compare the list items.

The instructor's example: the first render had a list of five elements: e1, e2, e3, e4, e5. After some logic runs, the new list is e1, e2, e6, e4, e5: the number of elements is the same but one middle element has a different value. Without keys React does not compare item by item. It rebuilds the whole list.

A list of plain strings is cheap. But imagine each item is a complicated card with an image, text and buttons, such as a list of e-commerce products or social media posts. Recreating the whole list when only one item changed (data updated, a new element inserted, or an existing element deleted) is **not efficient** and wastes time recreating elements that were already on the DOM.

With a key

With a unique key on each element, the virtual DOM compares the list of the previous DOM with the latest DOM, finds the exact differences, and **only that specific element is updated on the screen**. The entire list is not recreated.

	Without key	With unique key
Virtual DOM compares the list?	No identification factor, so no per-item comparison	Yes, item by item using the key
When one element changes	Whole list is recreated	Only that element is updated

	Without key	With unique key
Effect on large, complex lists	Slow, wasted work	Efficient
React console message	Error/warning: each child should have a unique key	No message

The instructor said the lag may not be visible in a tiny practice application, but in real-time applications with lots of data and components the difference is real. A good developer always passes the key.

10.3 Student questions: uniqueness and tables without a unique column

- **Why must the key be unique?** If you give duplicates, React again cannot tell elements apart and cannot compare them. So the key must be unique.
- **What if the table has no unique column?** Then you have to create one. The instructor added: if a table has no unique column, put a question mark on your design. Something is wrong with the design. Every system should have something unique: an id, or a combination such as id plus name. If everything is the same in two rows, introduce something new that identifies each row uniquely.
- The key concept is **not limited to class or function components**; it is for any component. Wherever you render a list, pass a key.

ADDITIONAL EXPLANATION (not from the lecture)
The following is bonus context, not from the lecture. In the key's absence React silently uses the **array index** as a fallback and still shows the warning. Using the index as a key is also risky when the list can be re-ordered, filtered or have items inserted, because the identity of an item changes with its position. A stable id from your data (like `user.id`) is the best key. The lecture's simplified explanation ("the whole list is recreated") expresses the performance concern in an easy way.

COMMON MISTAKE
Do not use `Math.random()` or a new value on every render as a key. A key must stay the same for the same item across renders and different for different items.

11. Performance Optimisation and Memoization

There are many ways to optimise the performance of a React application. The one introduced now is **memoization**.

11.1 The multiplication analogy

Suppose I give you a task: multiply **352 by 922** with pen and paper, no calculator, no device. It takes you around 2 to 3 minutes. That is an **expensive calculation**. The answer on your paper is **324544**.

CLARIFICATION OF THE TRANSCRIPT
The transcript first says 923, then 922, and the spoken result is garbled. 352 x 922 = **324,544**. The numbers are only an example; the idea is what matters.

Now I ask the **same question again**: what is 352 x 922? You have two options: calculate again (another 2-3 minutes) or **look at the paper** where you wrote the first answer (one second). Everyone answered: read the stored result. If the input changes, say to a different pair of numbers, you must calculate again. That is fine.

FROM THE LECTURE
Memoization is simply this: if an expensive calculation is needed repeatedly and the **input has not changed**, do not recalculate. The first time, do the calculation and **cache (store) the result** somewhere in memory. From the second time onwards, if the same input is given, **return the cached result** instead of doing the calculation again.

11.2 What memoization does NOT do

- It does **not** make the first calculation faster. If a calculation takes 3 minutes of CPU time, the first run still takes that time. To make that faster you would need different hardware (or other tools).
- The instructor mentioned that speeding up heavy work (for example, by running it in the background) is a job for **Web Workers** in JavaScript, a topic to look at when you find time. It is not memoization.
- Memoization only avoids **repeating the same work for the same input**.

11.3 With and without memoization in a React application

	Without memoization	With memoization
Same input, repeated request	Recalculates every time	Returns the cached result
Different input	Recalculates (correct)	Recalculates (correct)
Cost when input is unchanged	Full cost every time	Almost nothing

Memoization can be used for two things in React: (1) **caching the results** of expensive calculations or operations, and (2) **stopping unnecessary re-rendering of components**. As a beginner you build components and they work, but behind the scenes needless re-renders can make the application feel slow or laggy. The next sections show where that happens and how to avoid it.

12. Parent/Child Components and Unnecessary Re-rendering

12.1 What are parent and child components?

The top-level component that contains another component is the **parent**. The component rendered inside it is the **child**. In the project: `App` renders `UsersData`, so `App` is the parent and `UsersData` is its child. Likewise inside `UsersData` the `div`, `h2` and `table` are its children. Every component that renders another component is a parent of it.

12.2 Building the demo

Create `Greeting.jsx` (the child) and `Parent.jsx` (the parent). The child receives a prop `name` and shows "Hello name". The instructor destructured the props: `const { name } = props`. If `name` is Aman it shows "Hello Aman"; if Rahul, "Hello Rahul".

Code 12.1 - Greeting child with a console.log

```
// Greeting.jsx  (child component, normal version)
function Greeting(props) {
  const { name } = props;                       // take out the value of name
  console.log("Greeting child component rendered");
  return (
    <div>
      <h4>Hello {name}</h4>
    </div>
  );
}

export default Greeting;
```

Code 12.2 - Parent with a state and a button

```
// Parent.jsx
import { useState } from "react";
import Greeting from "./Greeting";

function Parent() {
  const [count, setCount] = useState(0);

  const increase = () => {
    setCount(count + 2);                         // lecture: adds 2 on each click
  };
```

```
  return (
    <div>
      <h3>Count is {count}</h3>
      <button onClick={increase}>Increase</button>
      <Greeting name="Aman" />                    {/* hard-coded prop */}
    </div>
  );
}

export default Parent;
```

12.3 Observing the problem

Render `Parent` inside `App`. On the first load the console shows the child message once (fine). Now click the button repeatedly.

- Which state is updated on click? The **parent's** state, because the parent created it.
- Parent state updated -> the parent re-renders (obvious).
- But the **child also re-renders every time**, and the console message from `Greeting` appears again and again.
- Is anything changing in the child? **No.** The prop passed is always the same `"Aman"`.

> **FROM THE LECTURE**
> When the prop value for the child is the same all the time, there is no need to re-render the child. That re-render is **unnecessary rendering**. The re-render is **necessary** only when the prop passed to the child actually changes because of the parent's state update. (For example, if you passed `count` as the prop, a re-render of the child is needed because its data changed.)

In a tiny component the cost is invisible. But think of a large child such as a **dashboard**, which may call APIs and do a lot of work; re-rendering it for the same data re-executes all of that. The instructor asked the class to stop this for the same prop value, and said the tool for it is in the next part (after a half-hour break).

Situation	Re-render the child?
Parent state changes, child's props are the same	No - this is unnecessary re-rendering
Parent state changes, child's props changed	Yes - necessary re-rendering

13. React.memo - Memoizing a Component

After the break the instructor recapped: one parent, one child with a prop; a state in the parent; a button updates the parent state; parent re-renders and so does the child, although the child's props did not change. To stop this unnecessary re-rendering, use the predefined function `memo` from React.

13.1 What memo does

> **FROM THE LECTURE**
> `memo` takes your **entire component as the argument** and returns a **memoized version** of that component. You are not just memoizing a result, you are memoizing the whole component. Before re-rendering a memoized child, React **checks the props**: are they the same as in the previous render? If yes, **no re-render**. If the props changed, it re-renders. The React documentation line the instructor showed: it lets you skip re-rendering a component when its props are unchanged.

13.2 Two ways to write it

Way 1 - two steps (pass the component to memo, export the result)

Code 13.1 - memo in two steps

```
import { memo } from "react";

function Greeting(props) {
  const { name } = props;
  console.log("Greeting child component rendered");
  return <h4>Hello {name}</h4>;
}

export const MemoizedGreeting = memo(Greeting);   // memoized component
```

Way 2 - one step using an arrow function (the instructor's final version)

Code 13.2 - memoized Greeting (arrow function wrapped in memo)

```
import { memo } from "react";

export const Greeting = memo((props) => {
  const { name } = props;
  console.log("Greeting child component rendered");
  return (
    <div>
      <h4>Hello {name}</h4>
    </div>
  );
});
```

In `Parent.jsx`, import `{ Greeting }` (a named export now) instead of the default import, and use `<Greeting name="Aman" />` exactly as before.

13.3 Output and explanation

- First load: parent and child both render once (console shows the child message once).
- Click the button: the parent state updates and the parent re-renders.
- Because the prop is the same (`Aman`), the memoized child is **not** re-rendered. The child message does not appear again.
- If the child received a prop that depends on the parent's state, for example `count`, then it would re-render each time that prop changes.

13.4 Why memoize every component

A student (Mukul) asked whether every component should be memoized. The instructor said yes, it is always good to memoize components. In this demo there is only one level, but in real applications there are **many levels**: A renders B, B renders C, C renders D, D renders E. If A updates its state, the whole hierarchy re-renders unless memoized, and you will feel lag in the application. These small things prevent the lag.

ADDITIONAL EXPLANATION (not from the lecture)

Note beyond the lecture: `memo` itself has a small cost (it compares props on each parent render), so in very tiny components the benefit is negligible. The instructor's classroom advice is to adopt the habit of memoizing components; in practice teams often apply it where profiling shows re-render cost.

KEY POINT

`memo` -> memoizes an **entire component**; re-renders only when **props change** (shallow comparison). If a prop is a **function**, there is a catch, shown in Section 15.

14. useMemo - Memoizing the Result of a Calculation

14.1 Phase 1 - the problem (an expensive calculation inside a component)

Create `ExpensiveCalculation.jsx`. It has two states, `x` (starts at 10) and `y` (starts at 50). It also has a **local variable** `total` (not a state, not a prop) calculated with a loop that depends on `x`. The loop runs a very large number of times so that it is really slow (the instructor started at 1 crore and then added two more zeros so the delay became about 1 to 2 seconds). Two buttons change the states: **Change x** adds 5 to `x`, **Change y** adds 3 to `y`.

Code 14.1 - expensive calculation without memoization

```
import { useState } from "react";

function ExpensiveCalculation() {
  const [x, setX] = useState(10);
  const [y, setY] = useState(50);

  // expensive calculation: depends only on x
  let total = 0;
  for (let i = 1; i <= 1000000000; i++) {
    total = total + x;
  }

  const changeX = () => {
    setX(x + 5);
  };

  const changeY = () => {
    setY(y + 3);
  };

  return (
    <div>
      <h3>Total: {total}</h3>
      <h4>x = {x}, y = {y}</h4>
      <button onClick={changeX}>Change x</button>
      <button onClick={changeY}>Change y</button>
    </div>
  );
}

export default ExpensiveCalculation;
```

ADDITIONAL EXPLANATION (not from the lecture)

The loop count (1,000,000,000) is chosen to match the lecture's 1 to 2 second delay. On a slower computer it may take longer; reduce it by a few zeros if the page freezes.

What happens when you run it

- The component renders for the first time, so the loop runs (this is fine and required).
- Click **Change x**: `x` updates, the component re-renders, and the loop runs again - a 1-2 second delay. This is **logical**, because the result depends on `x`.
- Click **Change y**: `y` updates, the component re-renders, and **the loop runs again** with the same result because `x` did not change. The delay is visible again. This is wasted work.

FROM THE LECTURE

The delay seen when clicking **Change y** is the proof. A component re-render executes **everything** in the function body again, whichever state was updated. The expression is evaluated on **every render**: the first render, the second, the third... The component does not know that the calculation has nothing to do with `y`. Also, until the calculation finishes, the state update is not reflected on the screen, so the page waits.

Question the instructor asked: which state update should run the recalculation and which should not? Answer (class): when `x` updates, recalculate - valid. When `y` updates, do not recalculate. This is the "input has not changed, so do not recalculate" scenario from memoization, and the hook for it is `useMemo`.

14.2 Phase 2 - the improved approach: `useMemo`

`useMemo` **memoizes the result of any expression, calculation, operation or logic**, not a whole component. The result may come from a loop, a string concatenation, an arithmetic or mathematical calculation, or any business logic. Its arguments are the same shape as `useEffect`: a **callback function** and a **dependency array**.

- Put the calculation inside the callback.
- **Return** the final result (here, `total`) from the callback.
- In the dependency array, pass the states the calculation depends on. Here it depends on `x`, so write `[x]`.
- `useMemo` **returns** the memoized value. The instructor stored it in a variable named `optimizedTotal` and displayed that variable.

Code 14.2 - the same component with useMemo

```
import { useMemo, useState } from "react";

function ExpensiveCalculation() {
  const [x, setX] = useState(10);
  const [y, setY] = useState(50);

  const optimizedTotal = useMemo(() => {
    let total = 0;
    for (let i = 1; i <= 1000000000; i++) {
      total = total + x;
    }
    return total;                // result to be memoized
  }, [x]);                       // recalculate only when x changes

  const changeX = () => {
    setX(x + 5);
  };

  const changeY = () => {
    setY(y + 3);
  };

  return (
    <div>
      <h3>Total: {optimizedTotal}</h3>
      <h4>x = {x}, y = {y}</h4>
      <button onClick={changeX}>Change x</button>
      <button onClick={changeY}>Change y</button>
    </div>
  );
}

export default ExpensiveCalculation;
```

How it works, in execution order

1. **First render:** x = 10. `useMemo` runs the loop, gets the total, **stores** it and returns it (`optimizedTotal`).
2. **Click Change x:** x becomes 15. On re-render `useMemo` compares the current `x` (15) with the previous `x` (10). They differ, so it **recalculates** and stores the new result. You see the 1-2 second delay.
3. **Click Change y:** only `y` changes. `useMemo` checks `x`: same as the last render, so it does **not** recalculate and simply **returns the cached value**. No lag, which is the proof.

Why does the output appear this way?

The loop is now inside `useMemo`, which executes the callback only when a value in the dependency array has changed. For all other renders it returns the stored result directly.

14.3 Dependency array mistakes (very important)

What you write	Effect	Result
`[x]` (correct)	Recalculates only when x changes	Fast, correct
`[x, y]` (extra dependency)	Recalculates when y changes too	The lag returns - unnecessary calculation. IDEs may warn: useMemo has an unnecessary dependency y
Missing dependency (e.g. `[]` when x is used)	Never recalculates; always returns the cached data	Bug: the operation looks fine but the UI does not show the changed result

COMMON MISTAKE

The instructor: "The dependency array must be passed very carefully." Forgetting to pass a dependency state means the result is **never** recalculated and you keep seeing old data, even though the state has really changed. Adding a dependency you do not need makes an **unnecessary calculation** run again. Check that you passed the **correct** dependencies and not any extras.

14.4 Student questions on useMemo

- **Is useMemo the same as componentDidMount/didUpdate?** (Om) No. Mount/update are lifecycle phases that let you control **what to do after** a state update. `useMemo` is used to **memoize a calculation result** and to decide which updates should or should not re-run the calculation.
- **Can we fetch data inside useMemo?** (Mukul) Not recommended. Write the data-fetch in a separate function. `useMemo` should only be one level of abstraction that memoizes a **result**; it does not care where data came from. Fetching code should not go inside `useMemo`.
- **Should we memoize API responses?** Usually not. The browser already caches responses, and server-side caching can use tools such as **Redis**. `useMemo` is generally used to cache the result of **your own business logic**.
- **Business-logic example:** the API returns `firstName` and `lastName` separately but the UI must show them together. Call the API normally, and put the concatenation logic in `useMemo` so it is recalculated only when the data state changes.

KEY POINT

Instructor's habit: analyse each component; any calculation, evaluation or business logic that computes a value -> put it in `useMemo`.

15. Function Props, Reference Equality and useCallback

The next requirement: how to memoize when a **function** is passed as a prop. Same parent/child relationship, but the layout is different this time.

15.1 Setup: state in the parent, button in the child

- `Counter.jsx` is the **parent** and holds the state `count` (initial 0) and the function `increaseCount`.
- `CounterChild.jsx` is the **child**; it contains the button that should increase the count.
- So the state and function are in the parent, but they must be **called from the child**. A parent can pass data **and functions** to a child using props.
- The prop is named `plusClickHandler`. Its value is the **reference** of `increaseCount` (the function is not called while passing, only referred to).

Code 15.1 - child: receives a function in props and attaches it to the button

```
// CounterChild.jsx
import { memo } from "react";
```

```
export const CounterChild = memo((props) => {
  const { plusClickHandler } = props;          // destructure the function prop
  console.log("Counter child component re-rendered");
  return (
    <div>
      <button onClick={plusClickHandler}>Plus</button>
    </div>
  );
});
```

Code 15.2 - parent: state, function and the child

```
// Counter.jsx  (parent) - first version, NOT optimised
import { useState } from "react";
import { CounterChild } from "./CounterChild";

export function Counter() {
  const [count, setCount] = useState(0);

  const increaseCount = () => {
    setCount(count + 10);                      // lecture: increases count by 10
  };

  return (
    <div>
      <h2>Count = {count}</h2>
      <CounterChild plusClickHandler={increaseCount} />
    </div>
  );
}
```

Render `Counter` in `App.jsx`. On click of Plus the count updates correctly. The instructor summarised: we are **updating the parent's state from the child**. The flow is simple:

- Button is in the child, function and state are in the parent.
- Parent -> passes function via props -> child receives it (named `plusClickHandler`) -> the button's `onClick` refers to it.
- A prop can be a number, a string, an array, **or a function**.

15.2 Phase 2 - the hidden problem

The child is already memoized with `memo`. So you would expect it to re-render only when its props change. Yet, when the Plus button is clicked, the console shows the child re-rendered. In the earlier Greeting example, with a hard-coded prop, memo worked. What is different?

> **FROM THE LECTURE**
> In the previous example the prop was a fixed value ("Aman"). This time the prop is a **function**. **Functions are objects in JavaScript** - everything is an object. Every time a component renders, **a new reference is created for each function** defined inside it. When `memo` compares old and new props, for primitive values it compares the **values**, but for a function it compares the **reference**, since a function has no comparable value. The reference changes on every render, so the props always look different, and the memoized child re-renders anyway.

The instructor's memory-address illustration

Render	`increaseCount` content	Reference (illustrative)	Is it the same as before?
1st render	`setCount(0 + 10)`	#123abcd	-
Re-render (e.g. another state changed)	`setCount(0 + 10)` - same code	#456def	No -> child re-renders

Note that the memory addresses above are made-up values used only to explain the idea. The definition (the code) of the function is the same, but its **reference** is new; `memo` compares references, not definitions, so it sees a difference.

KEY POINT

Rule: when a function is passed in props, the reference is compared (not the value). Every render creates a new reference, so `memo` alone cannot stop re-rendering. A student (Om) summarised this well: primitives (string, number) are compared by value; functions and objects are compared by reference.

15.3 Showing the problem clearly with a second state

Add another state `message` (initial `"Hello"`) in the parent, a button **Change message**, and a function `changeMessage` that sets message to the current value + "bye" (so you see it change every time). Show the count and the message on screen.

Code 15.3 - extra state that has nothing to do with the child

```
const [message, setMessage] = useState("Hello");

const changeMessage = () => {
  setMessage(message + " bye");
};

// in the JSX of the parent
<h3>Message: {message}</h3>
<button onClick={changeMessage}>Change message</button>
```

Experiment: click **Change message**. The child re-renders, even though `count` and the `increaseCount` code did not change at all. Reason: the message state changed -> parent re-rendered -> `increaseCount` **was recreated with a new reference** -> `memo` saw a different prop -> child re-rendered. The instructor's point: only a change in `count` should give the child a new function, because only then does the content of the function change.

15.4 Phase 3 - the solution: useCallback

What should we memoize? Not a result (so not useMemo), not the whole component (already done with memo). We must memoize the **function itself**. The hook for that is `useCallback`.

FROM THE LECTURE

Do not tell the child to "keep the reference the same". Instead tell the **function** to create a new reference **only when its dependency has changed**. `useCallback` is a barrier on re-creation: until the dependency changes, it does not allow a new reference of the function to be created and keeps returning the old one. It has the same shape as `useMemo`: a callback and a dependency array.

Code 15.4 - increaseCount wrapped in useCallback

```
// Counter.jsx  (parent) - optimised
import { useCallback, useState } from "react";
import { CounterChild } from "./CounterChild";

export function Counter() {
  const [count, setCount] = useState(0);
  const [message, setMessage] = useState("Hello");

  // memoize the FUNCTION; new reference only when count changes
  const increaseCount = useCallback(() => {
    setCount(count + 10);
  }, [count]);

  const changeMessage = () => {
    setMessage(message + " bye");
  };

  return (
    <div>
      <h2>Count = {count}</h2>
      <h3>Message: {message}</h3>
      <button onClick={changeMessage}>Change message</button>
      <CounterChild plusClickHandler={increaseCount} />
    </div>
```

```
    );
  }
```

Step-by-step explanation of how it behaves

1. **First render:** `useCallback` stores the function (this copy has `count = 0`, so it is like `setCount(0 + 10)`) in its cache, with a reference such as `#123abcd`. The function is **not run**; only stored.
2. **Click Change message:** `message` changes, parent re-renders. `useCallback` checks the dependency `count`: it did not change, so **no new function is created**; the old cached reference `#123abcd` is returned.
3. The memoized `CounterChild` compares old and new props: the reference is the **same**, so the child is **not re-rendered**.
4. **Click Plus:** `count` becomes 10. On the re-render `useCallback` sees that `count` changed, so it **creates a new function** (now the definition uses the new count) and caches that. The child gets a different reference and re-renders. This is required: the function must see the new count.

Expected output: clicking Plus -> count increases and the child re-renders (console message). Clicking Change message -> the message updates but the child **does not** re-render.

COMMON MISTAKE

Dependency array mistakes apply here too. If you leave `[]` empty, the first cached function (with `count = 0`) is used forever, so the count would always become 0 + 10 = 10 (a stale value) - that is the bug the instructor described. Always pass the states the function uses.

ADDITIONAL EXPLANATION (not from the lecture)

Bonus, not from the lecture: React also lets you write `setCount((prev) => prev + 10)`, which does not need `count` in the dependency array. The lecture sticks to `[count]` so that the dependency idea is clear.

16. memo vs useMemo vs useCallback

Feature	`memo`	`useMemo`	`useCallback`
What it is	A function (higher-order wrapper)	A hook	A hook
What it memoizes	The **entire component**	The **result** of a calculation / expression / logic	The **function itself**
Takes	The component	Callback + dependency array	Callback (the function) + dependency array
Returns	A memoized component	The memoized **value**	The memoized **function**
Recomputes / re-renders when	Props change	A dependency changes	A dependency changes (a new function reference)
Problem it solves	Unnecessary re-render of child	Repeated expensive calculation	New function reference on every render breaking memo
Where used	Wrap the child (and every component)	Inside the component, around calculations	Inside the component, around functions passed as props

16.1 The instructor's best practices

- **Every component** -> wrap it in `memo`.
- **Every calculation / expression / business-logic result** -> wrap it in `useMemo`.
- **Every function** -> wrap it in `useCallback`.
- **Dependencies matter most.** Wrong dependencies give either unnecessary renders (extra dependency) or stale data on the UI (missing dependency).

FROM THE LECTURE

Why: "you never know how many components below will re-render tomorrow". In real, complex dashboards, the hierarchy can be 15 to 20 levels deep; one small state update in a parent re-renders everything below and performance suffers. You may then see lots of API calls in the browser's network tab: that is a symptom of repeated re-rendering. The plan for the course: when the project is built (later with a UI library such as MUI), every component will be memoized, every function will be wrapped in `useCallback` and every business-logic result in `useMemo`.

CLARIFICATION OF THE TRANSCRIPT

The transcript sometimes says "mem-orization" or "memorise" and sometimes "memoize". The correct term is **memoization** (to memoize), meaning caching a result for reuse.

17. Doubt-Clearing Discussion

17.1 Passing data from child to parent

A student asked whether data can also be passed from child to parent. The instructor said yes, it can be done and will be covered step by step later. (The class had already seen the child calling a parent's function, which updates the parent state.)

17.2 How to start a project (advice given)

A student asked where to start when building projects: database, backend or frontend? The instructor's answer, in order:

1. **Think about the business entities first.** Not the front end and not the database yet. Example: in a hospital management system the entities are doctor, patient, OPD data, medicine, prescription.
2. Draw an **ER (entity relationship) diagram** and decide the relations: one-to-one, one-to-many, many-to-many (for example, one doctor has many patients).
3. **Design the database tables** from the ER diagram.
4. Decide **which operations the UI needs**: login, show products, update, delete, dashboard data.
5. **Create the APIs** for those needs (login APIs and dashboard APIs).
6. Finally build the **front end**.

17.3 How to learn hooks like useMemo and useCallback

The instructor said that he too did not understand these hooks at first because online articles give technically correct definitions but do not show what happens **with** and **without** the hook. His advice: recall what was taught, implement it yourself, watch videos and read articles if needed. Do not rely on a single source of truth; look at multiple sources and you will understand better.

ADDITIONAL EXPLANATION (not from the lecture)

The rest of the session was light conversation (weekend plans, work-life jokes, AI and careers). It contains no technical lecture content and is intentionally left out of the notes.

18. Revision Material

18.1 Concise summary of all major topics

- **Conditional rendering with loading:** use a separate `isLoading` state (start `true`, set to `false` in the same `setState` that stores the data). Check `isLoading` first, then `users.length`.
- **Class lifecycle:** constructor -> render -> componentDidMount (after first render) -> componentDidUpdate (after each re-render from state update) -> componentWillUnmount (just before removal); componentDidCatch handles errors (with ErrorBoundary).
- **Function components** have no native lifecycle; the single hook `useEffect(callback, dependencyArray)` replaces them.
- `[]` -> didMount; `[x, y]` -> didUpdate; returned function -> willUnmount. On the first render **all** effects run.
- Effect callbacks must return **only a function**; never mark them `async`; use an inner async function.
- **key prop:** required and unique for list items; helps React's virtual DOM update only changed items.
- **Memoization** = cache results; do not repeat the same work when input is unchanged.
- **Unnecessary re-render:** parent state change re-renders children even if their props are unchanged -> fix with `memo`.
- **useMemo** caches a calculated value; **useCallback** caches a function (needed because function props get a new reference each render).

18.2 Important definitions

Term	Definition / explanation	Purpose / key point
Lifecycle	The phases a component goes through: mounting, updating, unmounting (and error handling)	Lets you run code at each phase
Mounting	Component appears on the screen for the first time	Where data fetching on load is done
Unmounting	Component is removed from the screen	Cleanup work belongs here
Hook	A predefined React function that gives function components features such as state and lifecycle	`useState`, `useEffect`, `useMemo`, `useCallback`
Dependency array	Second argument of `useEffect`/`useMemo`/`useCallback` listing the values they depend on	Decides when the callback runs / recomputes
Conditional rendering	Showing different UI depending on a condition	Loading, no data, data views
Virtual DOM	React's own copy of the DOM used to compare previous and latest versions	Updates only what changed
key	Special prop that uniquely identifies each element of a rendered list	Efficient list updates
Memoization	Caching the result of a computation for reuse with the same input	Performance optimisation
Unnecessary re-render	A component re-renders although none of its props or states changed	Wasted time; avoid with `memo`
Reference	The identity (memory address) of an object or function	Functions and objects are compared by reference
Fallback UI	A simple UI shown when a component crashes	Used with componentDidCatch / ErrorBoundary

18.3 Syntax cheat sheet

```
// useEffect - didMount
useEffect(() => { /* run once after first render */ }, []);

// useEffect - didUpdate
useEffect(() => { /* run on first render and when x or y changes */ }, [x, y]);

// useEffect - willUnmount
useEffect(() => {
  return () => { /* cleanup before component is destroyed */ };
}, []);

// async inside useEffect (correct way)
useEffect(() => {
  async function load() { const res = await fetch(url); setData(await res.json()); }
  load();
}, []);

// key prop
{items.map((item) => <li key={item.id}>{item.name}</li>)}

// memo
export const Child = memo((props) => { return <div>{props.name}</div>; });

// useMemo
const result = useMemo(() => { return heavyCalculation(x); }, [x]);

// useCallback
const handler = useCallback(() => { setCount(count + 10); }, [count]);
```

18.4 Key differences between related concepts

Concept A	Concept B	Difference
`componentDidMount`	`componentDidUpdate`	Mount: only after the first render. Update: after later re-renders from state updates, not after the first.
`useEffect(cb, [])`	`useEffect(cb, [x])`	Empty array: after first render only. With x: first render and whenever x changes.
Class lifecycle methods	`useEffect`	Separate native methods vs one hook configured differently.
`isLoading` state	`users.length`	Is the request in progress vs do we have data.
`memo`	`useMemo`	Whole component vs result of a calculation.
`useMemo`	`useCallback`	Memoizes a value returned from a function vs memoizes the function itself.
Primitive prop	Function prop	Compared by value vs compared by reference (new reference each render).
Necessary re-render	Unnecessary re-render	Props changed vs props the same.

18.5 Common mistakes to avoid

Mistake	Why it is wrong	Correct approach
Deciding loading from `users.length === 0`	Empty server response shows Loading forever	Use a separate `isLoading` state
Two `setState` calls for loading and data	Unneeded; the instructor prefers one	One `setState({ users, isLoading: false })`
Writing a JS condition inside JSX without `{ }`	Not valid JSX	Wrap in curly braces
`<h4>text<h4/>`	Closing slash in wrong place	`<h4>text</h4>`
Writing `componentDidMount()` as a plain function inside a function component	Lifecycle methods are not native there	Use `useEffect`
Returning a number/string from a `useEffect` callback	Only a function may be returned	Return a cleanup function or nothing
`useEffect(async () => ...)`	Async function returns a promise	Inner async function called from the effect
Forgetting `return` for cleanup	Cleanup is not registered	Return a function from the effect
Thinking the whole effect runs on unmount	Only the returned function does	Put cleanup in the returned function
Missing `key` or duplicate keys	React cannot identify items; list is recreated	Unique, stable key such as `id`
Missing dependency in `useMemo`/`useCallback`	Stale data; UI not updated	Pass every state used
Extra dependency	Unnecessary recalculation or re-creation	Pass only what is needed
Passing a function prop to a memo child without `useCallback`	New reference each render defeats memo	Wrap the function in `useCallback`
Fetching data inside `useMemo`	Not recommended	Separate function; memoize only business logic results

18.6 Interview questions and answers

Questions marked **(lecture)** come directly from what the instructor taught. The marked **(extra)** ones go slightly beyond it.

Question	Answer
Name the lifecycle methods of a class component. (lecture)	constructor, render, componentDidMount, componentDidUpdate, componentWillUnmount, componentDidCatch.
When does componentDidMount run? (lecture)	Only once, just after the first render. It is used to fetch data at component load.
How is componentDidUpdate different from componentDidMount? (lecture)	didMount runs only after the first render; didUpdate runs after the re-renders that follow a state update and not after the first render.
What is componentWillUnmount used for? (lecture)	It is called just before the component is destroyed; cleanup like cancelling pending API requests and releasing resources goes here.
What does componentDidCatch do? (lecture)	It handles errors that occur in the component. With ErrorBoundary it lets the app show a fallback UI instead of crashing entirely.
Are lifecycle methods available in function components? (lecture)	No. They are native to class components. In function components we use the `useEffect` hook.
How do you implement didMount, didUpdate and willUnmount with useEffect? (lecture)	Empty dependency array for didMount; dependencies in the array for didUpdate; return a function from the callback for willUnmount.
Which useEffects run on the first render? (lecture)	All of them, whatever their dependency arrays.
Can the useEffect callback return a value? (lecture)	Only a function (used for cleanup). Anything else gives an error.
Why can't you make the useEffect callback async? (lecture)	An async function returns a promise, but the callback may return only a function. Create an inner async function and call it.

Question	Answer
Why is the key prop needed in lists? (lecture)	It uniquely identifies each element so the virtual DOM can update only the changed element instead of recreating the entire list.
What is memoization? (lecture)	Caching the result of an expensive calculation and reusing it when the input has not changed.
What does memo do? (lecture)	Wraps a component and returns a memoized component that skips re-rendering when its props have not changed.
Difference between useMemo and useCallback? (lecture)	useMemo memoizes the result (value) of a calculation; useCallback memoizes a function so its reference stays the same until a dependency changes.
Why does a memoized child still re-render when you pass a function? (lecture)	Functions are objects; every render creates a new reference and memo compares references, so it sees a new prop. Fix with useCallback.
What happens if you forget a dependency in useMemo/useCallback? (lecture)	You keep using the old cached value or function, so the UI does not show the latest data.
Should you memoize API responses with useMemo? (lecture)	Generally no. Browsers and server caches (like Redis) handle that. useMemo is for your own business logic results.
Does memo make the first calculation faster? (lecture)	No. It only avoids repeating the same calculation for the same input.
Why use a separate isLoading state? (extra, built on the lecture)	Because the length of the data cannot tell you whether the request is still running; the two questions need two states.

18.7 MCQ-style points to remember

- The second argument of `useEffect` is the **dependency array**.
- An empty dependency array gives **componentDidMount** behaviour.
- On the **first render** every `useEffect` runs.
- The `key` prop must be **unique** within a list.
- `memo` compares **props**; for functions it compares **references**.
- `useCallback` memoizes a **function**; `useMemo` memoizes a **value**.
- The error text for `return 7` says an effect must not return anything besides a function.

18.8 Practice exercises (solutions not included)

Easy

- Write down the order in which a class component's constructor, render, componentDidMount, componentDidUpdate and componentWillUnmount are called.
- Create a function component with a state `name` and a `useEffect` with an empty array that logs a message. What do you see in the console on load?
- Add a `key` to a list of products rendered with `map`. What happens to the console warning?
- Explain in two sentences what memoization is, using your own example (not multiplication).

Medium

- Convert the class `UsersList` into a function component, with an `isLoading` state and the three UI cases: loading, users not found, rows.
- Create two states `a` and `b` and two `useEffect`s: one with `[]` and one with `[a]`. Predict, then check, the console output when you click buttons that change `a` and `b`.
- Write a `useEffect` that starts a `setInterval` and stops it when the component unmounts. Switch between two components to see the cleanup run.

- Rewrite a `useEffect` that was written with `async () =>` so that it follows React's rule.

Challenging

- Build a parent with a `count` state and a `Child` that gets a hard-coded prop. Add logs and prove the child re-renders; fix it with `memo`; then pass `count` as a prop and observe the new behaviour.
- Create an expensive calculation depending on `x` only. Measure the delay with and without `useMemo`; then purposely add `y` to the dependency array and explain the result.
- Make a `Counter`/`CounterChild` pair: pass a function with and without `useCallback`; explain with references why the logs differ.
- Introduce a bug on purpose by leaving `count` out of the `useCallback` dependency array. Describe the stale behaviour you see.

18.9 Final revision checklist

[ ] I can explain why `users.length` alone is a wrong loading indicator and how `isLoading` fixes it.

[ ] I can write nested conditional rendering for loading / empty / data states in JSX without syntax errors.

[ ] I can name all class lifecycle methods and say when each runs.

[ ] I understand the scoreboard use case for componentDidUpdate and the cancel-request use case for componentWillUnmount.

[ ] I know why lifecycle is not native in function components and that `useEffect` is the hook for it.

[ ] I can write useEffect for didMount, didUpdate and willUnmount from memory.

[ ] I know that all useEffects run on the first render and which ones run later.

[ ] I know that the effect callback may return only a function and must not be async.

[ ] I can fetch data in a function component with an inner async function and show it in a table with `map`.

[ ] I always add a unique `key` when rendering lists and I can explain the virtual DOM reason.

[ ] I can explain memoization with an everyday example.

[ ] I can show unnecessary re-rendering in a parent/child pair and fix it with `memo`.

[ ] I can use `useMemo` with the correct dependency array and explain what goes wrong with extra or missing dependencies.

[ ] I can explain why function props defeat `memo` and how `useCallback` fixes it.

[ ] I can state the roles of `memo`, `useMemo` and `useCallback` in one sentence each.

[ ] I know the project order: entities/ER diagram -> database -> APIs -> front end.

19. Completeness Check Against the Transcript

The transcript was read from start to finish and a checklist of topics was made. Each topic is matched to a section of these notes.

Topic discussed in the lecture (in order)	Covered in
Recap of last session, new demo app, setup commands	Section 1
constructor, render, componentDidMount recap	Section 1.4
UsersList class component recap; 3-second delay and Loading message	Sections 1.5, 2.1
Hidden bug: empty array shows Loading forever	Section 2.2
isLoading state and single setState call	Section 2.3
Nested conditions, tbody syntax error, closing-tag error, Users not found	Sections 2.3, 2.4

Topic discussed in the lecture (in order)	Covered in
componentDidUpdate with scoreboard example	Section 3.1
componentWillUnmount, Java finalize analogy, cancelling API calls, garbage collector question	Section 3.2
componentDidCatch, error boundary, fallback UI, navbar/counter/footer example	Section 3.3
Sequence summary of class lifecycle	Section 3.4
Lifecycle not native in function components; hooks; useEffect as the single hook	Section 4
useEffect syntax (callback + dependency array)	Section 4.3
useEffect as componentDidMount; UsersData with then/catch; useState	Section 5
useEffect as componentDidUpdate; x, y, z example; student doubt	Section 6
Multiple useEffects; first-render rule; buttons and console demo; why keep both effects	Section 7
useEffect as componentWillUnmount; return a function; return 7 error; comparison of three variations	Section 8
Function-component fetch with async/await; why async on the callback fails; fetchUsersData; table and map	Section 9
key prop: rule, why, virtual DOM, e1-e6 example, uniqueness, no unique column	Section 10
Memoization concept with the multiplication example, web workers mention, caching, unnecessary re-rendering	Section 11
Parent/child definition; Greeting and Parent demo; unnecessary re-render	Section 12
memo function, two ways to write, memoize every component, deep hierarchy	Section 13
ExpensiveCalculation demo; useMemo; dependency mistakes; what not to put in useMemo; Redis/browser caching; first name + last name example	Section 14
Counter / CounterChild; function props; reference comparison; message state; useCallback	Section 15
memo vs useMemo vs useCallback; best practices	Section 16
Child-to-parent data question; project order advice; learning advice	Section 17
Revision material (summary, definitions, syntax, differences, mistakes, interview Q&A, exercises, checklist)	Section 18

KEY POINT

The prompt's DOM topics (getElementById, innerHTML, onclick, form validation and so on) are for JavaScript lectures. This lecture is a React lecture and did not cover them, so they do not appear in these notes. Casual talk at the end (weekend, careers, jokes) was intentionally left out.